Package 'facereaderconverter'

October 13, 2026

Title Process of FaceReader outputs and emotional episodes

Version 0.4

Description Tools to convert FaceReader outputs; Markup of emotional episodes.

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

Depends R (>= 4.1.0)

Imports dplyr,
hms,
janitor,
readr,
readxl,
stringr,
tidyselect,
tools,
tibble,
Rcpp,
data.table,
tidyr,
openxlsx

LinkingTo Rcpp

Suggests testthat (>= 3.0.0)

Config/testthat/edition 3

Config/roxygen2/version 8.1.0

Contents

add_delta_column 2
assess_frame_drift 3
convertFRDirectory 4
convertFRExcelFiles 5
convertFRFiles 7
convert_directory_to_episodes 8
convert_to_episodes 10
delta 11
delta_episodes 12

export_shared_synchrony_clips . . . 13
extract_id_subject_metadata . . . 15
extract_subject_id_metadata . . . 16
infer_fps . . . 17
loadFRfile . . . 18
locf . . . 19
map_paths . . . 20
merge_episodes . . . 20
negative_controls . . . 21
parse_time_to_frame . . . 23
reaction_rate . . . 23
reaction_rate_by_episode . . . 25
shared_synchronous_episodes . . . 26
synchrony . . . 28
synchrony_by_episode . . . 30
synchrony_moments_pipeline . . . 31
to_seconds . . . 33

Index **35**

`add_delta_column` *Add delta column to coding_df*

Description

`add_delta_column()` is deprecated because `convert_to_episodes()` now computes and returns `delta` automatically.

Usage

```
add_delta_column(
  coding,
  delta = 0.1,
  delta_window = 0.2,
  fps = NULL,
  cores = 0L
)
```

Arguments

`coding`	Data frame with columns: id, subject, emotion, frame (or video_time), value, or an `fr_coding` object.
`delta`	Threshold for both up and down.
`delta_window`	Window size in seconds.
`fps`	Frames per second; may be non-integer. A single number applies to all ids, a named numeric vector or a data frame with columns `id` and `fps` sets it per id. Rates stored in an `fr_coding` object's metadata take precedence (a conflicting value warns). If omitted, it is inferred from `video_time`, falling back to 30.
`cores`	integer Number of threads to use. Default 0 is auto.

Value

coding_df with a delta column where 1 means up and 0 means down.

Examples

```
## Not run:
coding_df = read.csv("testdata_detailed.csv")
add_delta_column(
    coding_df,
    delta_window = 0.2,
    delta = 0.1,
    fps = 30
 )

## End(Not run)
```

---

assess_frame_drift *Assess drift between frame indices and video time stamps*

Description

For each id, compares the time implied by each frame index (frame / fps) with the parsed video_time in seconds. Frame indices are integers, so a drift of up to half a frame is expected when frame was derived by rounding; larger drift suggests an incorrect fps, dropped frames or irregular time stamps.

Usage

```
assess_frame_drift(x, fps = NULL)
```

Arguments

x	An fr_coding object, or a data frame with columns id, video_time and frame.
fps	Optional fps used for ids without stored metadata: a single number, a named numeric vector, or a data frame with columns id and fps. Stored metadata takes precedence (with a warning on conflict); the default is 30.

Value

A data.table with one row per id: fps, n (distinct time stamps), max_abs_drift_s, mean_abs_drift_s (seconds) and max_abs_drift_frames. A warning is issued if any id drifts by more than half a frame.

Examples

```
## Not run:
assess_frame_drift(converted, fps = 30)
assess_frame_drift(coding_df, fps = c("1" = 29.97, "2" = 30))

## End(Not run)
```

convertFRDirectory *Convert a directory of Facereader files to CSV*

Description

Reads TXT, XLSX, and CSV files in a folder and converts them to CSV.

Usage

```
convertFRDirectory(
  inpath,
  outpath = inpath,
  recursive = TRUE,
  pattern = NULL,
  values_as_numeric = TRUE,
  clean_names = TRUE,
  save_metadata = outpath,
  metadata_filename = "metadata.csv",
  fail_codes = FALSE,
  duplicate_timecodes_as_error = TRUE,
  cores = 0L,
  ...,
  id = NULL,
  subject = NULL,
  id_pattern = NULL,
  subject_pattern = NULL,
  use_full_path = FALSE
)
```

Arguments

inpath	Path to an existing directory.
outpath	Path to save the csvs to defaults to the inpath
recursive	Bool as to whether to look for all files in directory (TRUE) or just the root folder (FALSE)
pattern	a regex pattern of files to test, if NULL then will look for TXT, XLSX, and CSV files
values_as_numeric	Save values as numeric, where applicable
clean_names	returns janitor-style clean names
save_metadata	save the metadata as a csv in the outpath, set to NULL to not save
metadata_filename	filename of the metadata csv
fail_codes	adds a column with the fail reason, True or False. Column then has 0 for success, 1 for fit_failed, 2 for find_failed
duplicate_timecodes_as_error	throws an error if there are duplicate timecodes, if FALSE then throws warning
cores	integer Number of threads to use. Default 0 is auto.

...	arguments passed as necessary
id	Optional scalar ID or function of each input path returning one. If omitted, TXT/XLSX inputs use the associated media filename without its extension; CSV inputs have no inferred ID. Output CSV filenames only include id/subject when they, or their patterns, are supplied.
subject	Optional scalar subject or function of each input path returning one. If omitted, TXT/XLSX inputs use the input filename without its extension; CSV inputs only infer a subject when subject_pattern is supplied.
id_pattern, subject_pattern	Optional regular expressions for inferring metadata from the media filename and FaceReader export filename.
use_full_path	If TRUE, search full paths when patterns are supplied.

Value

Invisibly returns the metadata.

Examples

```
## Not run:
convertFRDirectory(
  inpath="directory_of_txt_files",
  outpath="directory_to_save_csvs_to",
  values_as_numeric = TRUE
)

## End(Not run)
```

---

convertFRExcelFiles *Load FaceReader Excel exports*

Description

Reads a FaceReader .xlsx file, detects the header row automatically, and returns the parsed data frame. The function follows the same post-processing conventions as convertFRFiles() where applicable.

Usage

```
convertFRExcelFiles(
  inpath,
  return_data = TRUE,
  values_as_numeric = TRUE,
  clean_names = TRUE,
  fail_codes = FALSE,
  duplicate_timecodes_as_error = TRUE,
  sheet = 1,
  ...,
  id = NULL,
  subject = NULL,
  id_pattern = NULL,
```

```
  subject_pattern = NULL,
  use_full_path = FALSE
)
```

Arguments

`inpath`	Path to an existing `.xlsx` file.
`return_data`	Return the parsed data instead of metadata.
`values_as_numeric`	Convert `Video Time` to `hms` and detailed values to numeric.
`clean_names`	Apply `janitor::clean_names()` to the data.
`fail_codes`	Add a `fail_code` column for detailed exports.
`duplicate_timecodes_as_error`	Throw an error if duplicate timecodes are found.
`sheet`	Excel sheet to read. Defaults to the first sheet.
`...`	Additional arguments passed to `janitor::clean_names()`.
`id`	Optional scalar ID or function of `inpath` returning one. If omitted, the associated media filename without its extension is used.
`subject`	Optional scalar subject or function of `inpath` returning one. If omitted, the FaceReader filename without its extension is used.
`id_pattern, subject_pattern`	Optional regular expressions for inferring metadata from the media filename and FaceReader export filename.
`use_full_path`	If `TRUE`, search full paths when patterns are supplied.

Value

Invisibly returns the parsed data when `return_data = TRUE`; otherwise returns metadata about the import.

Examples

```
## Not run:
convertFRExcelFiles(
  inpath = "FaceReaderOutput.xlsx",
  return_data = TRUE,
  values_as_numeric = TRUE
)

## End(Not run)
```

`convertFRFiles` *Convert Facereader TXT to CSV*

Description

Reads a .txt file, detects the FaceReader header row automatically, guesses the delimiter from the header line, writes a .csv with the same basename in the same directory, and returns the path to the created CSV (invisibly).

Usage

```
convertFRFiles(
  inpath,
  outpath = paste0(tools::file_path_sans_ext(inpath), ".csv"),
  return_data = FALSE,
  values_as_numeric = TRUE,
  clean_names = TRUE,
  fail_codes = FALSE,
  duplicate_timecodes_as_error = TRUE,
  ...,
  id = NULL,
  subject = NULL,
  id_pattern = NULL,
  subject_pattern = NULL,
  use_full_path = FALSE
)
```

Arguments

`inpath`	Path to an existing .txt file.
`outpath`	Path to save the csv to defaults to the inpath
`return_data`	Bool to return the data from the txt rather than the metadata
`values_as_numeric`	Save values as numeric, where applicable
`clean_names`	returns janitor-style clean names
`fail_codes`	adds a column with the fail reason, True or False. Column then has 0 for success, 1 for fit_failed, 2 for find_failed
`duplicate_timecodes_as_error`	throws an error if there are duplicate timecodes, if FALSE then throws warning
`...`	arguments passed as necessary
`id`	Optional scalar ID or function of `inpath` returning one. If omitted, the associated media filename without its extension is used. The output CSV filename only includes `id`/`subject` when they, or their patterns, are supplied.
`subject`	Optional scalar subject or function of `inpath` returning one. If omitted, the FaceReader filename without its extension is used.
`id_pattern, subject_pattern`	Optional regular expressions for inferring metadata from the media filename and FaceReader export filename.
`use_full_path`	If `TRUE`, search full paths when patterns are supplied.

Value

Invisibly returns the metadata.

Examples

```
## Not run:
convertFRFiles(
  inpath="FaceReaderOutput.txt",
  values_as_numeric = TRUE
)

## End(Not run)
```

`convert_directory_to_episodes` *Convert a directory of detailed FaceReader exports to episodes*

Description

Recursively loads detailed TXT and XLSX exports, combines their emotion values, and calls `convert_to_episodes()` once. Each export's frame rate is inherited from its header (non-integer rates such as 29.97 are kept) and stored per `id` in `metadata$fps`; frame rates may differ between ids but exports that resolve to the same `id` must agree. When a header has no usable frame rate it is inferred from the time stamps with `infer_fps()`. State exports are ignored. Returns the combined result in memory and saves it as `coded_data` in one `.RDa` file.

Usage

```
convert_directory_to_episodes(
  inpath,
  outpath = file.path(inpath, "episodes.RDa"),
  id_pattern = NULL,
  subject_pattern = NULL,
  recursive = TRUE,
  overwrite = FALSE,
  T_up = 0.2,
  T_down = 0.1,
  delta = 0.1,
  delta_window = 0.2,
  min_dur_sec = 0.1,
  consecutive_missing = 150L,
  cores = 0L,
  use_full_path = FALSE,
  skip_fails = TRUE,
  filter_name = NULL
)
```

Arguments

inpath	Directory containing FaceReader exports.
outpath	Path to the output .RDa file. Defaults to episodes.RDa inside inpath.
id_pattern, subject_pattern	Optional regular expressions applied to the associated media filename and FaceReader export filename, respectively. If a rule is NULL, the complete corresponding basename without extension is used. A supplied subject_pattern must match every detailed export. An id_pattern (or media filename) that yields no match gives an NA ID; exports must still resolve to distinct ID and subject pairs, including NA IDs. Multiple ID matches in a media filename are rejected. Set use_full_path = TRUE to search the full media filename stored in metadata for IDs and the full FaceReader export path for subjects.
recursive	Whether to search subdirectories.
overwrite	Whether to replace an existing output file. If FALSE, an existing output triggers a warning and conversion is skipped. Defaults to FALSE.
T_up, T_down, delta, delta_window, min_dur_sec, consecutive_missing, cores	Passed to convert_to_episodes().
use_full_path	Search full metadata/export paths when a regex is supplied. Basename defaults are unchanged.
skip_fails	Whether to warn and skip individual exports that fail parsing, validation, or loading. Defaults to TRUE. An error is still raised if no detailed exports can be processed.
filter_name	Optional regular expression matched against the export filename (including its extension), not its directory path.

Value

The combined fr_coding object returned by convert_to_episodes(), invisibly. The same object is saved as coded_data in outpath. If outpath already exists and overwrite is FALSE, loads and invisibly returns its saved coded_data without converting.

Examples

```
## Not run:
coded_data <- convert_directory_to_episodes(
  "path/to/exports",
  id_pattern = "[0-9]{4}",
  subject_pattern = "mum|teen",
  filter_name = "_detailed\\.(txt|xlsx)$",
  skip_fails = TRUE
)

## End(Not run)
```

`convert_to_episodes` *Convert coding_df to episodes*

Description

Convert coding_df to episodes

Usage

```
convert_to_episodes(
  coding_df,
  T_up = 0.2,
  T_down = 0.1,
  delta = 0.1,
  delta_window = 0.2,
  min_dur_sec = 0.1,
  consecutive_missing = 150L,
  fps = NULL,
  cores = 0L
)
```

Arguments

`coding_df`	a dataframe or an `fr_coding` object from a FaceReader output. For an `fr_coding` object, its `$coding` table is reprocessed using the supplied parameters. id and subject should be present.
`T_up`	numeric Upper threshold for entering an episode. Default: 0.2.
`T_down`	numeric Lower threshold for exiting an episode. Default: 0.1.
`delta`	numeric Threshold used when computing the returned `delta` column, as the minimum k-step change required. It no longer controls episode boundaries. Default: 0.1.
`delta_window`	numeric Time window (in seconds) used to derive `k`, the lag for the returned k-step difference `delta` column. It no longer controls episode boundaries. Default: 0.1.
`min_dur_sec`	numeric Minimum episode duration (in seconds). Default: 0.1.
`consecutive_missing`	integer Maximum allowed consecutive missing (NA) frames while in-state before forcing episode end. Default: 150L.
`fps`	Frames per second (sampling rate of the data); may be non-integer (e.g. 29.97). A single number applies to all ids; a named numeric vector or a data frame with columns `id` and `fps` sets it per `id`. Used only for ids whose rate is not already stored in metadata (a warning is issued on conflict). If omitted, the rate is inferred from `video_time` (see `infer_fps`), falling back to 30. Data returned by `loadFRfile()` and the `convertFR*` functions carry the frame rate from the file header (as an `fr_metadata` attribute) and this is used before `fps`; supply `fps` only when the file has no frame rate. (Combining several files with e.g. `rbind()` drops the attribute; use `convert_directory_to_episodes()` for mixed rates.) A single shared rate is stored in `metadata$fps` as a scalar; mixed rates are stored as a table with columns `id` and `fps`.
`cores`	integer Number of threads to use. Default 0 is auto.

Details

Episode boundaries are determined only by `T_up` and `T_down`. The returned `delta` column is computed separately and delta-up rows are also returned in `deltas` for downstream functions such as `reaction_rate()`. The function uses the exported native binding `hysteresis_state` if available; otherwise it will error. It relies on **data.table** for fast grouping and joins.

Value

A list with four elements:

episodes data.table of detected episodes with columns `start_frame`, `end_frame`, `n_frames`, `duration_s`, `id`, `subject`, `emotion`, `start_time`, `end_time`, `run_id`, and `max_value`.

deltas data.table of delta-up reaction events with the same columns as `episodes`, except `delta_id` replaces `run_id`, and with `max_delta`, the value range from one delta window before the event start through the event end.

coding Annotated data.table containing the original columns plus `id`, `subject`, `emotion`, `value`, `delta`, `delta_id`, `run_id`, `status`, and `in_state`. `status` marks episode boundaries with `1L` at the start frame and `0L` at the end frame; `in_state` is `TRUE` for frames inside detected episodes.

metadata Metadata used to create the returned object.

Examples

```
## Not run:
coding_df = read.csv("testdata_detailed.csv")
convert_to_episodes(coding_df)

## End(Not run)
```

---

delta *Add delta column alias*

Description

`delta()` is deprecated because `convert_to_episodes()` now computes and returns `delta` automatically.

Usage

```
delta(coding, delta = 0.1, delta_window = 0.2, fps = NULL, cores = 0L)
```

Arguments

`coding`	Data frame with columns: id, subject, emotion, frame (or video_time), value, or an `fr_coding` object.
`delta`	Threshold for both up and down.
`delta_window`	Window size in seconds.
`fps`	Frames per second; may be non-integer. A single number applies to all ids, a named numeric vector or a data frame with columns `id` and `fps` sets it per id. Rates stored in an `fr_coding` object's metadata take precedence (a conflicting value warns). If omitted, it is inferred from `video_time`, falling back to 30.
`cores`	integer Number of threads to use. Default 0 is auto.

Value

coding_df with a delta column where 1 means up and 0 means down.

Examples

```
## Not run:
coding_df = read.csv("testdata_detailed.csv")
delta(
  coding_df,
  delta_window = 0.2,
  delta = 0.1,
  fps = 30
)

## End(Not run)
```

---

delta_episodes *Add create delta episodes*

Description

Add create delta episodes

Usage

```
delta_episodes(coding, fps = NULL, cores = 0L)
```

Arguments

coding	Data frame with columns: id, subject, emotion, frame (or video_time), value, or an fr_coding object
fps	Frames per second; may be non-integer. A single number applies to all ids, a named numeric vector or a data frame with columns id and fps sets it per id. Rates stored in an fr_coding object's metadata take precedence (a conflicting value warns). If omitted, it is inferred from video_time, falling back to 30.
cores	integer Number of threads to use. Default 0 is auto.

Value

episodes data.table of detected episodes with columns start_frame, end_frame,start_time, end_time, n_frames, duration_s, id, subject, emotion, and run_id.

Examples

```
## Not run:
coding_df = read.csv("testdata_detailed.csv")
x=add_delta_column(
    coding_df,
    delta_window = 0.2,
    delta = 0.1,
    fps = 30
 )
```

```
delta_episodes(x)


## End(Not run)
```

`export_shared_synchrony_clips` *Export clips for shared synchronous episodes*

Description

Selects shared synchronous intervals with the largest combined emotion values, or a named subject's highest-valued shared or individual episodes, and exports the corresponding video segments with FFmpeg. Filenames include the value-based selection rank and timestamp order among selected clips in each video. Frame ranges are inclusive: a range from frame 0 through frame 0 has duration `1 / fps`.

Usage

```
export_shared_synchrony_clips(
  coded_data,
  shared_synchrony,
  video_paths,
  n = 10L,
  emotion = "happy",
  optimised_subject = "both",
  only_synchronies = TRUE,
  buffer = NULL,
  buffer_units = c("seconds", "frames"),
  buffer_frames = 0L,
  buffer_seconds = 0,
  output_path = NULL,
  output = c("zip", "folder"),
  overwrite = FALSE,
  ffmpeg = "ffmpeg",
  verbose = FALSE,
  cores = 0L
)
```

Arguments

`coded_data`	A converted `fr_coding` object with `metadata$fps`. The frame rate may be a single number (applied to every id) or a table with one rate per `id`, in which case buffers, clip limits and start times are computed with each clip's own rate. Non-integer rates are allowed.
`shared_synchrony`	A table returned by `shared_synchronous_episodes()`.
`video_paths`	Named character vector of source video paths. Names must match values in the shared-synchrony `id` column.

n	Number of highest-ranked intervals to export. Use 0 to export all intervals remaining after filtering.
emotion	Optional character vector of emotions to retain.
optimised_subject	Subject-name regular expression used to rank clips by one subject's maximum emotion value. Defaults to "both", which ranks shared synchronies by their combined value.
only_synchronies	Logical; when TRUE, select shared synchronies. When FALSE, select individual episodes for optimised_subject. Ignored when optimised_subject = "both".
buffer	A non-negative number for a symmetric buffer, or a named two-element vector with names before and after for asymmetric buffers. Units are controlled by buffer_units. Defaults to 5 seconds before and 3 seconds after. Use NULL internally to select these defaults while preserving the legacy buffer arguments.
buffer_units	Buffer units: "seconds" or "frames".
buffer_frames	Non-negative whole-number symmetric frame buffer. Kept for compatibility; use buffer and buffer_units for new code.
buffer_seconds	Non-negative symmetric time buffer in seconds. Kept for compatibility; use buffer and buffer_units for new code.
output_path	Optional ZIP file path when output = "zip", or output directory when output = "folder". By default, output is placed beside the selected source video. All selected videos must then be in the same folder.
output	Output type: "zip" (the default) or "folder".
overwrite	Logical; overwrite an existing ZIP archive or non-empty output folder. Defaults to FALSE.
ffmpeg	Path to the FFmpeg executable or its command name.
verbose	Whether to display FFmpeg output instead of a clip progress bar.
cores	Maximum number of concurrent clip encoders. 0 (default) uses up to four physical cores, leaving one free; 1 runs sequentially.

Value

A data.table manifest invisibly. For ZIP output, the manifest is also included in the archive as manifest.csv.

Examples

```
## Not run:
coded_data <- convert_to_episodes(coding_data)
shared <- shared_synchronous_episodes(coded_data)
export_shared_synchrony_clips(
  coded_data,
  shared,
  video_paths = c("1" = "recording.mp4"),
  n = 10,
  optimised_subject = "child",
  only_synchronies = FALSE,
  buffer = c(before = 2, after = 1),
  output_path = "recording-shared-clips.zip"
```

```
)

## End(Not run)
```

`extract_id_subject_metadata` *Extract ID and subject metadata from a FaceReader filename*

Description

Compatibility alias for `extract_subject_id_metadata()`.

Usage

```
extract_id_subject_metadata(
  path,
  id_pattern = NULL,
  subject_pattern = NULL,
  use_full_path = FALSE
)
```

Arguments

`path`	Path to a FaceReader file.
`id_pattern`	Optional regular expression applied to the associated media filename. Defaults to the complete media basename without extension.
`subject_pattern`	Optional regular expression applied to the FaceReader export basename. Defaults to the complete basename without extension.
`use_full_path`	If `TRUE`, supplied patterns search the media filename as stored in the export for `id` and the full export path for `subject`. Default values remain basenames without extensions.

Value

A list with character elements `id` and `subject`. Each element is `NA` when its pattern does not match.

Examples

```
## Not run:
extract_id_subject_metadata("FR9 1218 mum_Analysis_detailed.xlsx")

## End(Not run)
```

`extract_subject_id_metadata` *Extract subject and ID metadata from a FaceReader filename*

Description

Extracts the media filename from a FaceReader export's metadata for `id` and the FaceReader export filename for `subject`. Both values are basenames without extensions unless an optional regular expression is supplied.

Usage

```
extract_subject_id_metadata(
  path,
  id_pattern = NULL,
  subject_pattern = NULL,
  use_full_path = FALSE
)
```

Arguments

`path`	Path to a FaceReader file.
`id_pattern`	Optional regular expression applied to the associated media filename. Defaults to the complete media basename without extension.
`subject_pattern`	Optional regular expression applied to the FaceReader export basename. Defaults to the complete basename without extension.
`use_full_path`	If `TRUE`, supplied patterns search the media filename as stored in the export for `id` and the full export path for `subject`. Default values remain basenames without extensions.

Value

A list with character elements `id` and `subject`. Each element is `NA` when its pattern does not match.

Examples

```
## Not run:
extract_subject_id_metadata("path/to/face_reader_export_detailed.txt")

## End(Not run)
```

infer_fps *Infer the frame rate of coded data from its time stamps*

Description

Estimates frames per second for each `id` when no frame rate is available from file metadata. For each `id` and `subject`, time stamps are sorted and the mean step between consecutive stamps is computed, ignoring gaps (steps longer than 1.5 times the median step, e.g. dropped frames). The per-subject estimates are combined by their median. Estimates within `tolerance` (relative) of a common rate are snapped to it; others are returned as estimated.

Usage

```
infer_fps(
  x,
  ids = NULL,
  tolerance = 0.005,
  standard_rates = c(24, 25, 29.97, 30, 50, 59.94, 60),
  min_rows = 10L
)
```

Arguments

`x`	A data frame with columns `id` and `video_time` (and optionally `subject`), or an `fr_coding` object (its `coding` table is used).
`ids`	Optional vector of ids to restrict the inference to.
`tolerance`	Relative tolerance for snapping to a common rate. Default 0.005.
`standard_rates`	Numeric vector of common frame rates to snap to.
`min_rows`	Minimum number of distinct time stamps needed for an estimate. Default 10.

Details

Warnings are issued when an `id` has too few time stamps, when subjects within an `id` disagree by more than `tolerance`, or when more than 5% of steps are gaps. Inference assumes regularly sampled time stamps; prefer the frame rate recorded in the file when it exists.

Value

A data.table with one row per `id`: `fps` (snapped when possible, `NA` if it could not be inferred), `fps_raw` (unsnapped estimate), `n` (largest number of distinct time stamps for a subject), `gap_prop` (proportion of steps treated as gaps) and `snapped`.

Examples

```
## Not run:
infer_fps(coding_df)
infer_fps(coding_df, ids = 1)

## End(Not run)
```

`loadFRfile` *Load FaceReader files into memory*

Description

Reads a FaceReader export file into memory by dispatching to the existing TXT, Excel, or CSV readers based on file extension. The wrapper always returns the parsed data and never writes an output file.

Usage

```
loadFRfile(
  inpath,
  values_as_numeric = TRUE,
  clean_names = TRUE,
  fail_codes = FALSE,
  duplicate_timecodes_as_error = TRUE,
  sheet = 1,
  ...,
  id = NULL,
  subject = NULL,
  csv_args = list(),
  clean_names_args = list(),
  id_pattern = NULL,
  subject_pattern = NULL,
  use_full_path = FALSE
)
```

Arguments

`inpath`	Path to an existing FaceReader export file.
`values_as_numeric`	Convert `Video Time` to `hms` and detailed values to numeric where applicable.
`clean_names`	Apply `janitor::clean_names()` to the data.
`fail_codes`	Add a `fail_code` column for detailed exports.
`duplicate_timecodes_as_error`	Throw an error if duplicate timecodes are found.
`sheet`	Excel sheet to read when importing `.xlsx` files.
`...`	Legacy additional arguments passed to `janitor::clean_names()` for TXT and XLSX imports and to `readr::read_csv()` for CSV imports.
`id`	Optional scalar ID or function of `inpath` returning one. If omitted, TXT/XLSX inputs use the associated media filename without its extension; CSV inputs have no inferred ID.
`subject`	Optional scalar subject or function of `inpath` returning one. If omitted, TXT/XLSX inputs use the input filename without its extension; CSV inputs only infer a subject when `subject_pattern` is supplied.
`csv_args`	Named list of additional arguments for `readr::read_csv()`.
`clean_names_args`	Named list of additional arguments for `janitor::clean_names()`.

id_pattern, subject_pattern
: Optional regular expressions for inferring metadata from the media filename and FaceReader export filename.

use_full_path
: If TRUE, search full paths when patterns are supplied.

Value

Invisibly returns the parsed data frame.

Examples

```
## Not run:
loadFRfile(
  inpath = "FaceReaderOutput.txt",
  values_as_numeric = TRUE
)

## End(Not run)
```

---

locf *Apply LOCF and Extract Episodes*

Description

Applies Last Observation Carried Forward (LOCF) logic to the coding data.table (from convert_to_episodes), imputing missing run_id values within blocks of non-missing value, grouped by id, subject, and emotion. Returns both the imputed coding table and a summary of contiguous episodes.

Usage

```
locf(coding, fps = NULL, consecutive_missing = NULL)
```

Arguments

coding
: A datatable, dataframe or fr_coding object as returned by convert_to_episodes, containing columns: id, subject, emotion, video_time, value, and run_id.

fps
: Frames per second; may be non-integer. A single number applies to all ids, a named numeric vector or a data frame with columns id and fps sets it per id. Rates inherited from an fr_coding object take precedence (a conflicting value warns). If omitted and not inherited, it is inferred from video_time, falling back to 30.

consecutive_missing
: Maximum allowed consecutive missing (NA) frames while in-state before forcing episode end. Default: 150L or inherited from an fr_coding object

Value

A list with four elements:

episodes data.table of contiguous episodes after LOCF, with columns `start_frame`, `end_frame`, `n_frames`, `duration_s`, `id`, `subject`, `emotion`, and `run_id`.

deltas data.table of delta-up events, passed through unchanged when the input is an `fr_coding` object.

coding Annotated data.table with LOCF-imputed `run_id`.

metadata Metadata from fr_coding object.

`map_paths` *Map files from an input root to an output root, preserve structure, create dirs.*

Description

Map files from an input root to an output root, preserve structure, create dirs.

Usage

```
map_paths(input_dir, output_dir, files)
```

Arguments

`input_dir`	Character scalar. The source root directory.
`output_dir`	Character scalar. The destination root directory.
`files`	Character vector. File paths under input_dir to be remapped.

Value

```
Character vector of output file paths (same length as files).
```

`merge_episodes` *Merge several fr_coding objects*

Description

Row-binds the `coding`, `episodes` and `deltas` tables of several `fr_coding` objects. The metadata is carried through once.

Usage

```
merge_episodes(x, envir = parent.frame())
```

Arguments

x	A list of fr_coding objects, or a character vector of names of fr_coding objects held in memory (the default way of supplying objects). A list may also contain symbols or names (e.g. list(quote(a), "b")), which are looked up in envir.
envir	Environment in which object names are looked up. Default: the calling environment.

Details

An error is thrown if the metadata of the objects differ in any way other than frame rate, or if any subject, id, emotion combination occurs in more than one object. Frame rates (metadata$fps) are combined per id: objects may use different rates for different ids and the result stores a per-id table (or a single value when all ids share one rate), but an error is thrown if the same id has different rates in different objects.

Value

An fr_coding object. run_id and delta_id are offset for each subsequent object so they stay unique, and remain consistent between coding, episodes and deltas.

Examples

```
## Not run:
a <- convert_to_episodes(coding_a)
b <- convert_to_episodes(coding_b)
merge_episodes(c("a", "b"))
merge_episodes(list(a, b))

## End(Not run)
```

---

negative_controls *Calculate synchrony for matched negative-control intervals*

Description

Samples control intervals with the same inclusive frame length as supplied episodes, then calculates synchrony for the sampled intervals. The episodes argument can be the output of synchrony_by_episode() or a compatible episode table. Control intervals cannot overlap supplied episodes or previously accepted controls for the same ID.

Usage

```
negative_controls(
  coded_data,
  episodes,
  mutually_exclusive = FALSE,
  subject = "subject",
  id = "id",
  time_limit = 3,
  time_limit_frames = NULL,
```

```
  constraint_method = "episode",
  fps = NULL,
  missing_threshold = 0,
  exclude_emotions = "neutral",
  max_tries = 1000L
)
```

Arguments

coded_data	Output from convert_to_episodes().
episodes	A data frame containing episode ranges. It must contain the selected ID column, either the selected subject column or denominator, and emotion, run_id, start_frame, and end_frame. Frame bounds are inclusive.
mutually_exclusive	Logical scalar retained for compatibility. Controls cannot overlap supplied episodes for the same ID regardless of this value.
subject	Character scalar giving the subject column. Defaults to "subject".
id	Character scalar giving the ID column. Defaults to "id".
time_limit	Maximum episode-window length in seconds. NULL is allowed when time_limit_frames is supplied or when constraint_method is "episode". Default is 3.
time_limit_frames	Optional maximum episode-window length in frames. If supplied, this takes precedence over time_limit.
constraint_method	Character scalar controlling how the episode window ends: "strict", "episode", "loose", or "frames". Default is "episode".
fps	Frames per second used to convert time_limit to frames; may be non-integer. A single number applies to all ids; a named numeric vector or a data frame with columns id and fps sets it per id. Rates stored in coded_data$metadata$fps take precedence (a conflicting value warns). If neither is available it is inferred from video_time, falling back to 30. time_limit is converted to frames separately for each id.
missing_threshold	Numeric scalar in [0, 1]. Denominator and numerator episodes are kept only when the comparison subject is present for at least this proportion of frames within the episode. Default is 0.
exclude_emotions	Character vector of emotions to exclude from the denominator calculation. Default is "neutral".
max_tries	Maximum number of random control starts attempted for each episode. Defaults to 1000.

Details

When coded_data contains metadata, its metadata$fps value is used by the synchrony calculations, including when fps is supplied.

Value

A data.table containing the usual synchrony_by_episode() columns for matched controls, plus source and control identifiers, frame bounds, attempt count, and matching status. Unmatched source episodes have missing synchrony fields and control_status = "unmatched".

Examples

```
## Not run:
coded_data <- convert_to_episodes(coding_data)
episodes <- synchrony_by_episode(coded_data)
negative_controls(coded_data, episodes)

## End(Not run)
```

`parse_time_to_frame` *Parse video time to frame index*

Description

Convert a time string into a frame index given frames-per-second (fps). Supports "HH:MM:SS", "MM:SS", or plain seconds and is vectorised over `video_time`.

Usage

```
parse_time_to_frame(video_time, fps)
```

Arguments

`video_time`	Character or numeric. Time strings in "HH:MM:SS", "MM:SS", or numeric seconds.
`fps`	Numeric. Frames per second.

Value

Integer vector. Frame indices computed as round(seconds * fps).

Examples

```
parse_time_to_frame("00:01:23.5", fps = 30)
parse_time_to_frame("1:23.5", fps = 30)
parse_time_to_frame("83.5", fps = 30)
```

`reaction_rate` *Calculate reaction rate from converted episodes*

Description

Reaction rate mirrors `synchrony()` but uses the numerator subject's delta-up events as the reaction signal. Denominator episodes come from `convert_to_episodes()`, and a reaction is counted when the numerator subject has at least one delta event within the constrained denominator episode window.

Usage

```
reaction_rate(
  coded_data,
  time_limit = 3,
  time_limit_frames = NULL,
  constraint_method = "episode",
  exclude_start = 0.1,
  exclude_start_frames = NULL,
  fps = NULL,
  subject_names = NULL,
  exclude_emotions = "neutral",
  minimum_threshold = 0
)
```

Arguments

coded_data	Output from convert_to_episodes(), which includes frame-level delta values in coded_data$coding and a reaction-event table in coded_data$deltas.
time_limit	Maximum episode-window length in seconds when a frame constraint is applied.
time_limit_frames	Optional maximum episode-window length in frames. If supplied, this takes precedence over time_limit.
constraint_method	Character scalar controlling how the reaction window ends. "strict" uses the earlier of the observed episode end and the requested time limit. "episode" uses the observed episode end and ignores time_limit. "loose" uses the later of the observed episode end and the requested time limit. "frames" uses only the requested time limit and ignores the observed episode end. Default is "episode".
exclude_start	Minimum delay, in seconds from the denominator episode start, before numerator delta == 1 values count as a reaction.
exclude_start_frames	Optional minimum delay in frames. If supplied, this takes precedence over exclude_start.
fps	Frames per second used to convert between seconds and frames; may be non-integer. A single number applies to all ids; a named numeric vector or a data frame with columns id and fps sets it per id. Rates in coded_data$metadata$fps take precedence (a conflicting value warns). If neither is available it is inferred from video_time, falling back to 30. time_limit and exclude_start are converted to frames separately for each id.
subject_names	Character vector of subject labels to compare. If NULL, all unique subjects in coded_data$coding are used.
exclude_emotions	Character vector of emotions to exclude from the denominator calculation. Default is "neutral".
minimum_threshold	Numeric scalar in [0, 1]. Denominator episodes are kept only when at least this proportion of numerator frames in the constrained window have non-missing values. Default is 0.

Value

A data.table with columns `id`, `denominator`, `numerator`, `emotion`, `n_episodes`, `n_reactions`, and `reaction_rate`.

See Also

`reaction_rate_by_episode()`

Examples

```
## Not run:
coded_data <- convert_to_episodes(coding_df)
reaction_rate(coded_data)

## End(Not run)
```

`reaction_rate_by_episode` *Calculate reaction rate by denominator episode*

Description

Calculate reaction rate by denominator episode

Usage

```
reaction_rate_by_episode(
  coded_data,
  time_limit = 3,
  time_limit_frames = NULL,
  constraint_method = "episode",
  exclude_start = 0.1,
  exclude_start_frames = NULL,
  fps = NULL,
  subject_names = NULL,
  exclude_emotions = "neutral",
  minimum_threshold = 0
)
```

Arguments

`coded_data`	Output from `convert_to_episodes()`, which includes frame-level `delta` values in `coded_data$coding` and a reaction-event table in `coded_data$deltas`.
`time_limit`	Maximum episode-window length in seconds when a frame constraint is applied.
`time_limit_frames`	Optional maximum episode-window length in frames. If supplied, this takes precedence over `time_limit`.

constraint_method
: Character scalar controlling how the reaction window ends. "strict" uses the earlier of the observed episode end and the requested time limit. "episode" uses the observed episode end and ignores time_limit. "loose" uses the later of the observed episode end and the requested time limit. "frames" uses only the requested time limit and ignores the observed episode end. Default is "episode".

exclude_start
: Minimum delay, in seconds from the denominator episode start, before numerator delta == 1 values count as a reaction.

exclude_start_frames
: Optional minimum delay in frames. If supplied, this takes precedence over exclude_start.

fps
: Frames per second used to convert between seconds and frames; may be non-integer. A single number applies to all ids; a named numeric vector or a data frame with columns id and fps sets it per id. Rates in coded_data$metadata$fps take precedence (a conflicting value warns). If neither is available it is inferred from video_time, falling back to 30. time_limit and exclude_start are converted to frames separately for each id.

subject_names
: Character vector of subject labels to compare. If NULL, all unique subjects in coded_data$coding are used.

exclude_emotions
: Character vector of emotions to exclude from the denominator calculation. Default is "neutral".

minimum_threshold
: Numeric scalar in [0, 1]. Denominator episodes are kept only when at least this proportion of numerator frames in the constrained window have non-missing values. Default is 0.

Value

A data.table with columns id, denominator, numerator, emotion, run_id, start_frame, end_frame, n_frames, present_prop, and reaction.

See Also

reaction_rate()

Examples

```
## Not run:
coded_data <- convert_to_episodes(coding_df)
reaction_rate_by_episode(coded_data)

## End(Not run)
```

---

shared_synchronous_episodes
Map shared synchronous episodes

Description

Maps pairs of same-emotion episodes that overlap while both subjects are in state. Each unordered pair of source episodes is returned once. A source episode may therefore occur in multiple rows when it overlaps more than one episode of the other subject.

Usage

```
shared_synchronous_episodes(
  coded_data,
  episodes = NULL,
  subject = "subject",
  id = "id",
  time_limit = 3,
  time_limit_frames = NULL,
  constraint_method = "episode",
  fps = NULL,
  missing_threshold = 0,
  exclude_emotions = "neutral"
)
```

Arguments

coded_data	Output from convert_to_episodes().
episodes	Optional replacement episode data frame. It must contain the selected ID and subject columns plus emotion, run_id, start_frame, end_frame, and max_value. Frame bounds are inclusive.
subject	Character scalar giving the column in coded_data$coding and coded_data$episodes that identifies the subject. Default is "subject".
id	Character scalar giving the column in coded_data$coding and coded_data$episodes that identifies the case or dyad. Default is "id".
time_limit	Maximum episode-window length in seconds. NULL is allowed when time_limit_frames is supplied or when constraint_method is "episode". Default is 3.
time_limit_frames	Optional maximum episode-window length in frames. If supplied, this takes precedence over time_limit.
constraint_method	Character scalar controlling how the episode window ends: "strict", "episode", "loose", or "frames". Default is "episode".
fps	Frames per second used to convert time_limit to frames; may be non-integer. A single number applies to all ids; a named numeric vector or a data frame with columns id and fps sets it per id. Rates stored in coded_data$metadata$fps take precedence (a conflicting value warns). If neither is available it is inferred from video_time, falling back to 30. time_limit is converted to frames separately for each id.
missing_threshold	Numeric scalar in [0, 1]. Denominator and numerator episodes are kept only when the comparison subject is present for at least this proportion of frames within the episode. Default is 0.
exclude_emotions	Character vector of emotions to exclude from the denominator calculation. Default is "neutral".

Value

A data.table with episode-pair identifiers, their inclusive shared frame interval, each source episode's maximum value, the larger maximum, and their combined value.

See Also

synchrony_by_episode()

Examples

```
## Not run:
coded_data <- convert_to_episodes(coding_data)
shared_synchronous_episodes(coded_data)

## End(Not run)
```

---

synchrony *Calculate synchrony from converted episodes*

Description

Calculate synchrony from converted episodes

Usage

```
synchrony(
  coded_data,
  subject = "subject",
  id = "id",
  time_limit = 3,
  time_limit_frames = NULL,
  constraint_method = "episode",
  fps = NULL,
  missing_threshold = 0,
  exclude_emotions = "neutral",
  episodes = NULL
)
```

Arguments

coded_data	Output from convert_to_episodes().
subject	Character scalar giving the column in coded_data$coding and coded_data$episodes that identifies the subject. Default is "subject".
id	Character scalar giving the column in coded_data$coding and coded_data$episodes that identifies the case or dyad. Default is "id".
time_limit	Maximum episode-window length in seconds. NULL is allowed when time_limit_frames is supplied or when constraint_method is "episode". Default is 3.
time_limit_frames	Optional maximum episode-window length in frames. If supplied, this takes precedence over time_limit.

`constraint_method`
: Character scalar controlling how the episode window ends: `"strict"`, `"episode"`, `"loose"`, or `"frames"`. Default is `"episode"`.

`fps`
: Frames per second used to convert `time_limit` to frames; may be non-integer. A single number applies to all ids; a named numeric vector or a data frame with columns `id` and `fps` sets it per id. Rates stored in `coded_data$metadata$fps` take precedence (a conflicting value warns). If neither is available it is inferred from `video_time`, falling back to 30. `time_limit` is converted to frames separately for each id.

`missing_threshold`
: Numeric scalar in `[0, 1]`. Denominator and numerator episodes are kept only when the comparison subject is present for at least this proportion of frames within the episode. Default is `0`.

`exclude_emotions`
: Character vector of emotions to exclude from the denominator calculation. Default is `"neutral"`.

`episodes`
: Optional replacement episode data frame. It must contain the selected ID and subject columns plus `emotion`, `run_id`, `start_frame`, and `end_frame`. Frame bounds are inclusive. A `synchrony_by_episode()` result is also accepted, using its `denominator` column as the episode subject.

Value

A data.table with columns `id`, `denominator`, `numerator`, `emotion`, `n_episodes`, and `synchrony`.

See Also

`synchrony_by_episode()`

Examples

```
library(data.table)

coding <- data.table(
  id = rep(1L, 6),
  subject = rep(c("teen", "parent"), each = 3),
  emotion = "happy",
  video_time = rep(1:3, 2),
  value = c(0.1, 0.2, 0.3, 0.1, 0.2, 0.3),
  in_state = c(FALSE, TRUE, FALSE, FALSE, FALSE, TRUE),
  run_id = c(1L, 1L, 1L, 2L, 2L, 2L)
)
episodes <- data.table(
  id = 1L,
  subject = c("teen", "parent"),
  emotion = "happy",
  run_id = c(1L, 2L),
  start_frame = c(2L, 3L),
  end_frame = c(2L, 3L)
)
coded_data <- structure(
  list(coding = coding, episodes = episodes),
  class = c("fr_coding", "list")
)
synchrony(coded_data, subject = "subject", id = "id", missing_threshold = 0)
```

synchrony_by_episode *Calculate synchrony by denominator episode*

Description

Calculate synchrony by denominator episode

Usage

```
synchrony_by_episode(
  coded_data,
  subject = "subject",
  id = "id",
  time_limit = 3,
  time_limit_frames = NULL,
  constraint_method = "episode",
  fps = NULL,
  missing_threshold = 0,
  exclude_emotions = "neutral",
  episodes = NULL
)
```

Arguments

coded_data	Output from convert_to_episodes().
subject	Character scalar giving the column in coded_data$coding and coded_data$episodes that identifies the subject. Default is "subject".
id	Character scalar giving the column in coded_data$coding and coded_data$episodes that identifies the case or dyad. Default is "id".
time_limit	Maximum episode-window length in seconds. NULL is allowed when time_limit_frames is supplied or when constraint_method is "episode". Default is 3.
time_limit_frames	Optional maximum episode-window length in frames. If supplied, this takes precedence over time_limit.
constraint_method	Character scalar controlling how the episode window ends: "strict", "episode", "loose", or "frames". Default is "episode".
fps	Frames per second used to convert time_limit to frames; may be non-integer. A single number applies to all ids; a named numeric vector or a data frame with columns id and fps sets it per id. Rates stored in coded_data$metadata$fps take precedence (a conflicting value warns). If neither is available it is inferred from video_time, falling back to 30. time_limit is converted to frames separately for each id.
missing_threshold	Numeric scalar in [0, 1]. Denominator and numerator episodes are kept only when the comparison subject is present for at least this proportion of frames within the episode. Default is 0.
exclude_emotions	Character vector of emotions to exclude from the denominator calculation. Default is "neutral".

episodes — Optional replacement episode data frame. It must contain the selected ID and subject columns plus emotion, run_id, start_frame, and end_frame. Frame bounds are inclusive. A synchrony_by_episode() result is also accepted, using its denominator column as the episode subject.

Details

When coded_data contains metadata, its metadata$fps value is used by the synchrony calculations, including when fps is supplied.

Value

A data.table with columns id, denominator, numerator, emotion, run_id, start_frame, end_frame, present_prop, and synchrony.

See Also

synchrony()

Examples

```
library(data.table)

coding <- data.table(
  id = rep(1L, 6),
  subject = rep(c("teen", "parent"), each = 3),
  emotion = "happy",
  video_time = rep(1:3, 2),
  value = c(0.1, 0.2, 0.3, 0.1, 0.2, 0.3),
  in_state = c(FALSE, TRUE, FALSE, FALSE, FALSE, TRUE),
  run_id = c(1L, 1L, 1L, 2L, 2L, 2L)
)
episodes <- data.table(
  id = 1L,
  subject = c("teen", "parent"),
  emotion = "happy",
  run_id = c(1L, 2L),
  start_frame = c(2L, 3L),
  end_frame = c(2L, 3L)
)
coded_data <- structure(
  list(coding = coding, episodes = episodes),
  class = c("fr_coding", "list")
)
synchrony_by_episode(coded_data, subject = "subject", id = "id", missing_threshold = 0)
```

---

synchrony_moments_pipeline

Create synchrony moments and video clips from FaceReader outputs

---

Description

Recursively discovers videos and FaceReader detailed TXT/XLSX exports below a folder. Each export is matched to a video using the basename in its FaceReader `Filename` metadata, ignoring case, directories, and file extensions. Videos without matching detailed exports are skipped with a message. Videos with other than two detailed outputs containing a single stable `Participant Name` are skipped with a warning. Repeated participant names across multiple outputs for one video are errors.

Usage

```
synchrony_moments_pipeline(
  inpath,
  output_dir = inpath,
  video_extension = "\\.(mp4|mov|mkv|avi)$",
  video_pattern = NULL,
  video_path = NULL,
  subject_from_filename = TRUE,
  subject_map = NULL,
  verbose = FALSE,
  T_up = 0.2,
  T_down = 0.1,
  delta = 0.1,
  delta_window = 0.2,
  min_dur_sec = 0.1,
  consecutive_missing = 150L,
  cores = 0L,
  time_limit = 3,
  time_limit_frames = NULL,
  constraint_method = "episode",
  missing_threshold = 0,
  exclude_emotions = "neutral",
  n = 10L,
  emotion = "happy",
  optimised_subject = "both",
  only_synchronies = TRUE,
  buffer = c(before = 5, after = 3),
  buffer_units = c("seconds", "frames"),
  output = c("zip", "folder"),
  overwrite = FALSE,
  ffmpeg = "ffmpeg"
)
```

Arguments

`inpath`	Existing directory containing videos and FaceReader exports.
`output_dir`	Optional root directory for clip outputs. Defaults to `inpath`.
`video_extension`	Regular expression matching supported video filename extensions. Defaults to common video formats.
`video_pattern`	Optional case-insensitive regular expression for video filenames. When supplied, only matching videos are considered.

video_path — Optional existing video path. When supplied, only this video is considered and both video filters are ignored.

subject_from_filename — Use the FaceReader export filename as the subject when its Participant Name column is absent.

subject_map — Optional named character vector mapping FaceReader export basenames, including extensions, to subject names. Matching ignores case; mapped names override Participant Name.

verbose — Whether to list videos with and without matching FaceReader detailed exports and display FFmpeg output. Videos without matching detailed exports are always reported as skipped; otherwise, clip progress is shown.

T_up, T_down, delta, delta_window, min_dur_sec, consecutive_missing, cores

Arguments passed to convert_to_episodes(). cores also controls concurrent clip encoders in export_shared_synchrony_clips(). The frame rate is read from each FaceReader export's Frame rate metadata, rounded to the nearest integer, and must agree within a video.

time_limit, time_limit_frames, constraint_method, missing_threshold, exclude_emotions

Arguments passed to shared_synchronous_episodes().

n, emotion, optimised_subject, only_synchronies, buffer, buffer_units, output, overwrite, ffmpeg

Arguments passed to export_shared_synchrony_clips().

Value

A synchrony_moments_pipeline list containing discovery manifest, all in-scope videos with match counts, frame rates, and status, and named per-video results for completed videos.

Examples

```
## Not run:
synchrony_moments_pipeline("path/to/study-folder", output_dir = "clips")

## End(Not run)
```

---

to_seconds *Convert FaceReader time strings to seconds*

Description

Parses time values formatted as hh:mm:ss or hh:mm:ss.s and returns the equivalent number of seconds. Fractional seconds are truncated or padded to the number of digits specified by digits.

Usage

```
to_seconds(x, digits = 3L)
```

Arguments

x	Character vector of time strings.
digits	Number of fractional digits to retain after the decimal point. Set to 0 to discard fractional seconds. Default is 3.

Value

A numeric vector of seconds.

Examples

```
## Not run:
to_seconds(c("00:00:10.000", "00:00:10.500"))

## End(Not run)
```

Index

add_delta_column, 2
assess_frame_drift, 3

convert_directory_to_episodes, 8
convert_to_episodes, 10
convert_to_episodes(), *8, 9*, 22, *33*
convertFRDirectory, 4
convertFRExcelFiles, 5
convertFRFiles, 7

delta, 11
delta_episodes, 12

export_shared_synchrony_clips, 13
export_shared_synchrony_clips(), *33*
extract_id_subject_metadata, 15
extract_subject_id_metadata, 16

infer_fps, *10*, 17
infer_fps(), *8*

janitor::clean_names(), *18*

loadFRfile, 18
locf, 19

map_paths, 20
merge_episodes, 20

negative_controls, 21

parse_time_to_frame, 23

reaction_rate, 23
reaction_rate(), *26*
reaction_rate_by_episode, 25
reaction_rate_by_episode(), *25*
readr::read_csv(), *18*

shared_synchronous_episodes, 26
shared_synchronous_episodes(), *13*, *33*
synchrony, 28
synchrony(), *31*
synchrony_by_episode, 30
synchrony_by_episode(), *21*, *22*, *28*, *29*
synchrony_moments_pipeline, 31

to_seconds, 33